- ***1*** Converts the floating-point numbers into a byte object to make it binary
- ***2*** This function takes neighbors in s and compares them to agent a; if they match, the reward is higher.

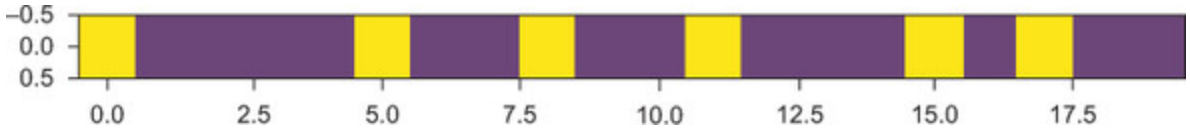
We have two functions in listing 9.3; the first creates a randomly initialized 1D grid (a vector) by first creating a grid of numbers drawn from a standard normal distribution. Then we set all the negative numbers to be 0 and all the positive numbers to be 1, and we will get approximately the same number of 1s and 0s in the grid. We can visualize the grid using matplotlib:

```
>>> size = (20,)
>>> grid = init_grid(size=size)
>>> grid
tensor([1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0,
0],
       dtype=torch.uint8)

>>> plt.imshow(np.expand_dims(grid,0))
```

As you can see in figure 9.12, the 1s are lightly shaded and the 0s are dark. We have to use the `np.expand_dims(...)` function to make the vector into a matrix by adding a singlet dimension, since `plt.imshow` only works on matrices or 3-tensors.

Figure 9.12. This is a 1D Ising model representing the electron spins for electrons arranged in a single row.



The second function in listing 9.3 is our reward function. It accepts a list, `s`, of binary digits, and a single binary digit, `a`, and then compares how many values in `s` match `a`. If all of the values match, the reward is maximal, and if none of them match, the reward is negative. The input `s` will be the list of neighbors. In this case, we will use the two nearest neighbors, so for a given agent its neighbors will be the agents to its left and right on the grid. If an agent is at the end of the grid, its right neighbor will be the first element in